

Міністерство освіти і науки України
Український державний університет науки і технологій
Кафедра ЕОМ

Лабораторна робота № 3
з дисципліни «Інформаційні технології в управлінській, науковій та викладацькій
діяльності»

Прийняв: доцент кафедри КІТ
Гончар А.М.
Виконав: студент групи 101М
Іваненко Іван Іванович

Дніпро
2026

Тема: Навчання моделі класифікації зображень за допомогою програми Lobe

Мета: набути практичних навичок навчання моделі класифікації зображень без написання програмного коду в програмі Lobe: підготувати й розмітити набір зображень, навчити модель, перевірити її на зображеннях, які не брали участі в навчанні, виправити помилкові прогнози, поліпшити модель за рахунок додаткових даних та експортувати її для використання в інших застосунках.

1 ВИХІДНІ ДАНІ

Модель має розрізняти чотири класи об'єктів: собаку (породи англійський спрингер-спанієль), церкву, сміттєвоз і парашут. Зображення взято з відкритого набору Imagenette – підмножини ImageNet, у якій менша сторона кожного зображення має 160 пікселів. Для кожного класу зібрано 90 зображень і розкладено їх у три теки, що не перетинаються (таблиця 1). Усередині кожної теки зображення лежать у підтеках з назвами класів, тому Lobe може розмітити їх автоматично.

Таблиця 1 – Склад набору зображень

Клас	навчання	доповнення	перевірка
собака	40	40	10
церква	40	40	10
сміттєвоз	40	40	10
парашут	40	40	10
Усього	160	160	40

Тека «навчання» використовується для першого навчання моделі, «доповнення» – для поліпшення моделі (п. 5.4 методичних вказівок), «перевірка» – для оцінки моделі на нових зображеннях (п. 5.3). До навчального набору з теки «перевірка» потрапили лише два зображення, мітки яких виправлено під час перевірки; у підрахунках це враховано окремо.

2 ВСТАНОВЛЕННЯ ПРОГРАМИ

Інсталяційний файл Lobe 0.11.714.2 отримано за посиланням викладача. Перед запуском перевірено його цифровий підпис: файл підписано Microsoft Corporation, підпис дійсний. Майстер установки пропонує теку C:\Program Files\Lobe, копіює файли програми, разом з нею встановлює Microsoft Visual C++ Runtime і завершується вікном з прапорцем «Run Lobe» (рисунок 1).

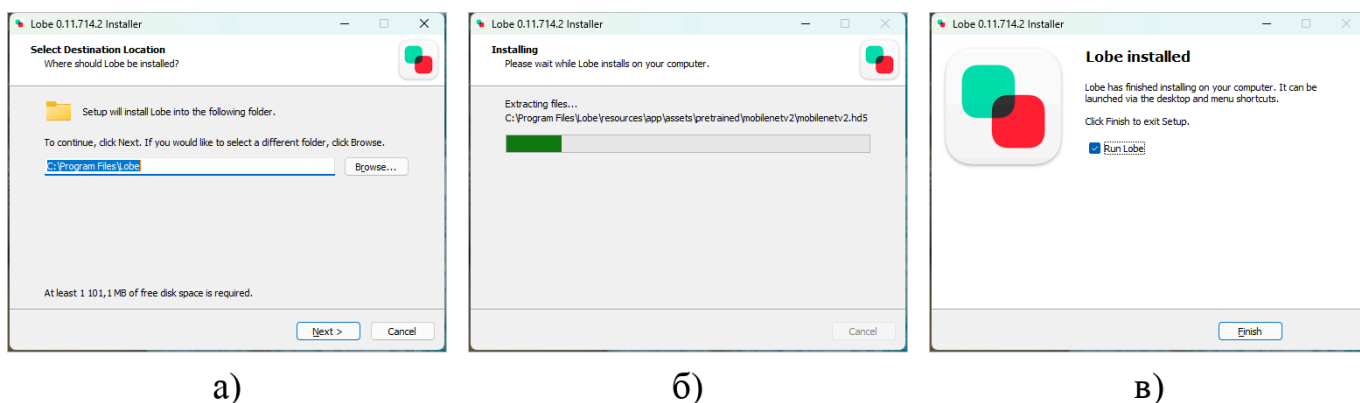


Рисунок 1 – Встановлення Lobe: а – вибір теки, б – копіювання файлів, в – завершення установки

Під час першого запуску програма показує умови ліцензійної угоди та положення про конфіденційність; згоду підтверджує кнопка «Continue». На пропозицію автоматично надсилати звіти про збої та аналітику обрано «Don't Share». Кнопка «Get Started» відкриває основний інтерфейс (рисунок 2). Брандмауер Windows при цьому запитав дозвіл на доступ до мережі для Lobe та його серверної частини Lobe Backend: програма працює через локальний сервер, до якого звертається і локальний API (розділ 7).

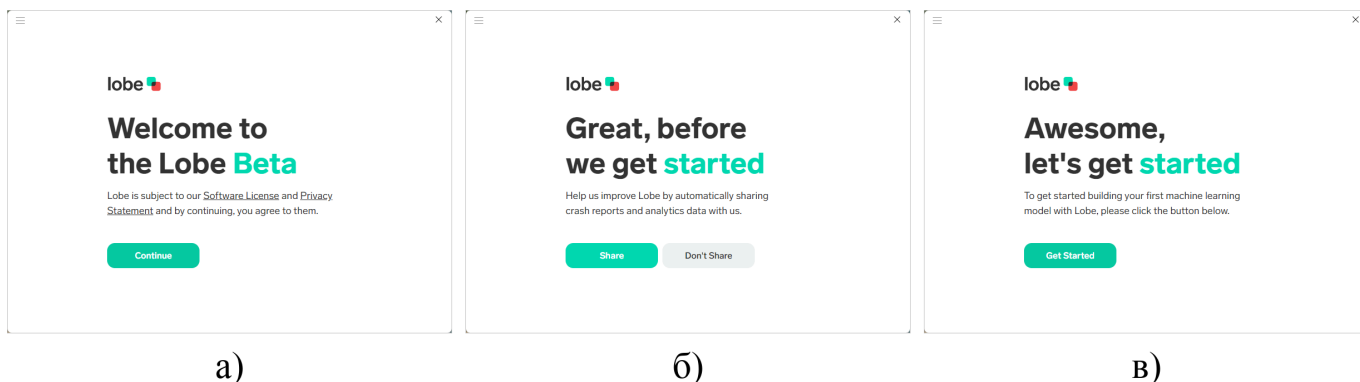


Рисунок 2 – Перший запуск: а – ліцензійна угода, б – відмова від надсилання аналітики, в – перехід до роботи

Після «Get Started» відкривається новий проєкт. Йому дано назву «Класифікація зображень» (рисунок 3). У лівій панелі розташовані три етапи роботи – Label (розмітка), Train (навчання) і Use (використання) – та перелік міток; поки зображень немає, програма нагадує, що для початку навчання потрібно щонайменше 5 зображень на мітку.

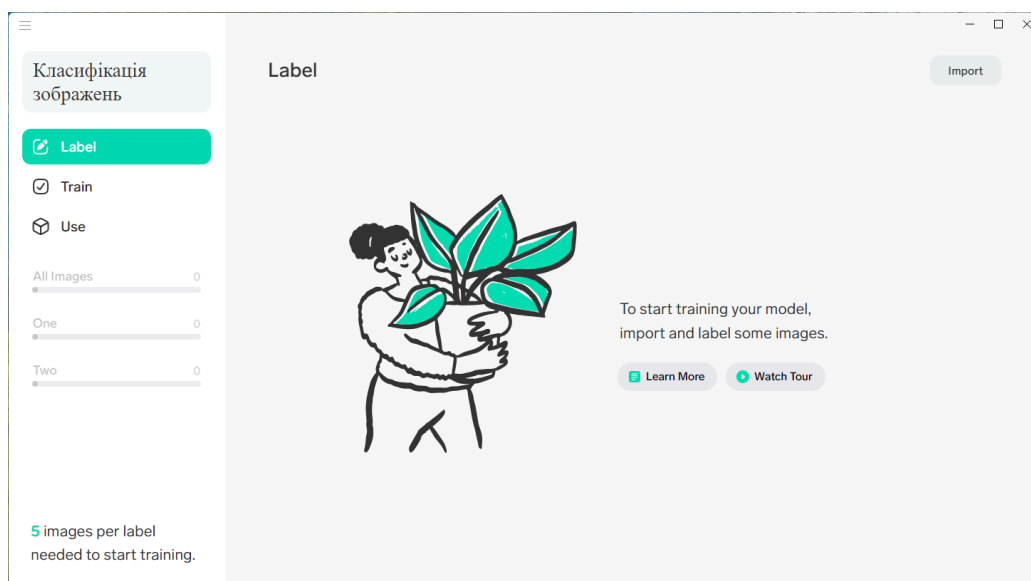


Рисунок 3 – Новий проєкт «Класифікація зображень»

3 ПІДГОТОВКА ТА РОЗМІТКА ДАНИХ

Кнопка «Import» пропонує три джерела зображень: файли з комп'ютера (Images), веб-камеру (Camera) і структуровану теку (Dataset) – рисунок 4. Оскільки зображення вже розкладено за класами, обрано Dataset.

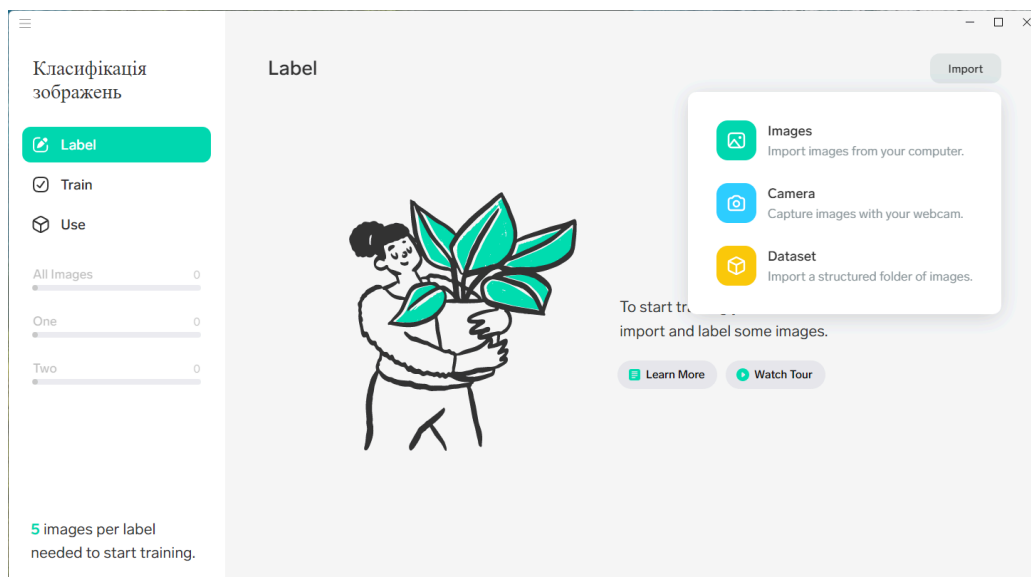


Рисунок 4 – Вибір джерела зображень

У вікні вибору теки вказано теку «навчання» (рисунок 5), після чого Lobe запитав, чи використовувати назви підтек як мітки (рисунок 6). Обрано «Label Using Folder Name»: усі 160 зображень отримали мітки без ручної розмітки, по 40 на кожную з чотирьох міток.

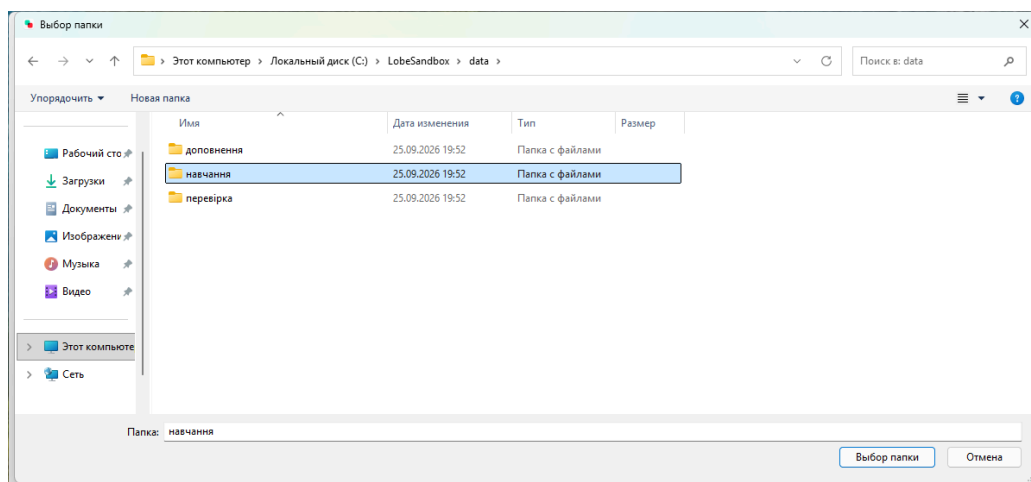


Рисунок 5 – Вибір теки з навчальними зображеннями

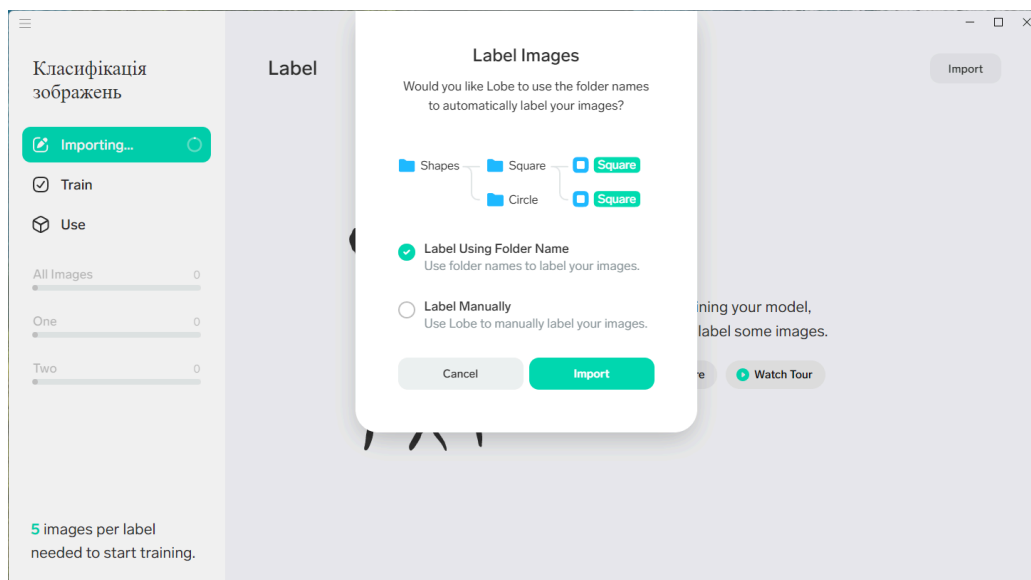


Рисунок 6 – Розмітка за назвами тек

4 НАВЧАННЯ МОДЕЛІ

Навчання почалося автоматично одразу після імпорту: на лівій панелі пункт Train змінився на «Training...» (рисунок 7). Lobe навчає модель на локальному комп'ютері; за журналом програми обчислення виконувалися через DirectML на відеокарті NVIDIA GeForce RTX 4080 SUPER, і навчання тривало близько 15 с. Графіків точності та втрат версія 0.11 не показує: динаміку видно за часткою правильно розпізнаних зображень під переліком міток, яка оновлюється під час навчання (61 % на рисунку 7).

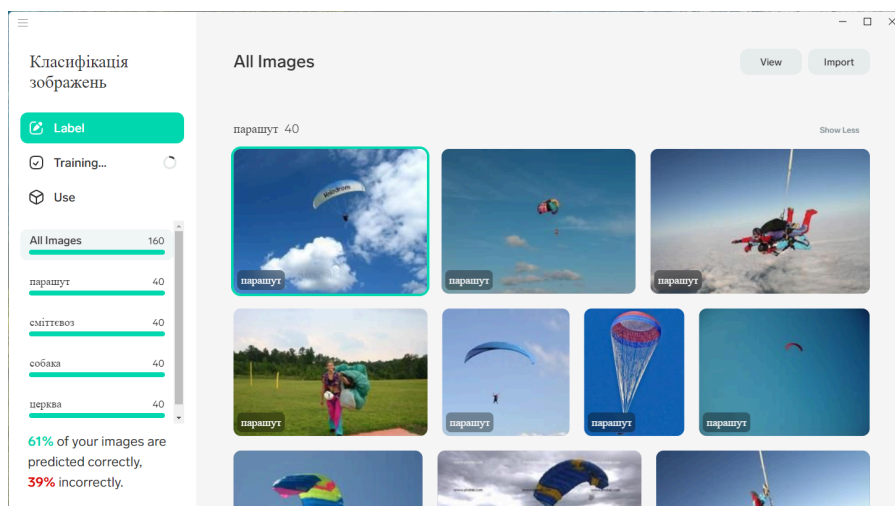


Рисунок 7 – Імпортовані зображення, триває навчання

За замовчуванням проєкт налаштовано на швидкодію (Optimize for Speed): Lobe донавчає компактну попередньо навчену мережу MobileNetV2. Після завершення навчання правильно розпізнано 91 % навчальних зображень (рисунок 8). Вкладка Train показує окремо правильно й помилково розпізнані зображення. Найгірший результат у класу «парашут» – 68 %: частину парашутів модель віднесла до сміттєвозів.

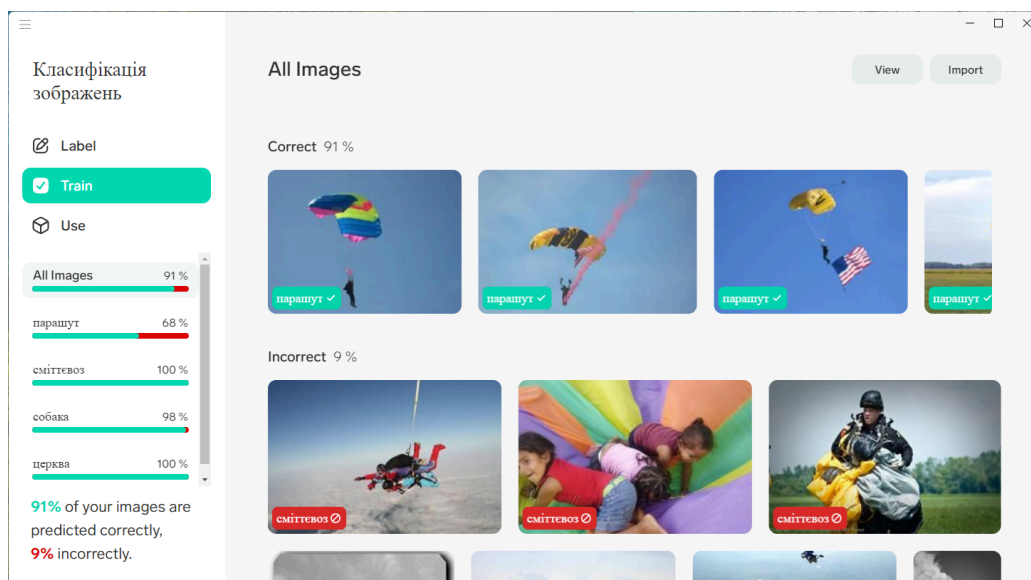


Рисунок 8 – Результат першого навчання

5 ПЕРЕВІРКА МОДЕЛІ

Для перевірки у вкладці Use (підвкладка Images) по одному відкривалися зображення з теки «перевірка». Під зображенням Lobe показує прогноз і дві кнопки: підтвердити прогноз або позначити його як помилковий. Церкву на рисунку 9 розпізнано правильно, парашут на рисунку 10 модель прийняла за церкву.

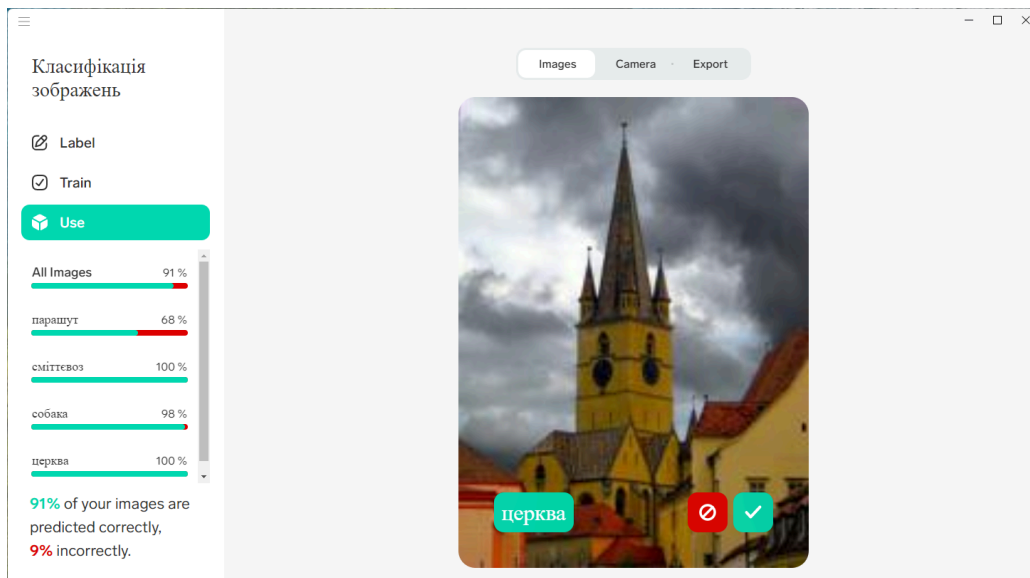


Рисунок 9 – Правильний прогноз на новому зображенні

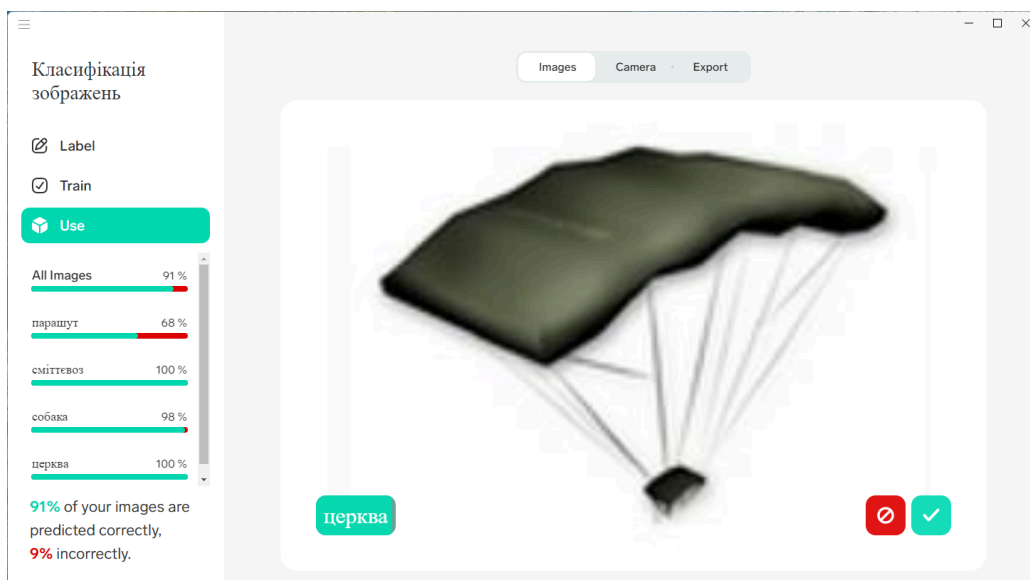


Рисунок 10 – Помилковий прогноз: парашут розпізнано як церкву

Щоб оцінити модель не за окремими прикладами, усі 40 зображень теки «перевірка» подано моделі через локальний API Lobe (розділ 7): кожне зображення надсиалося окремим запитом, прогноз порівнювався з назвою підтеки. Перша модель розпізнала правильно 34 зображення з 40 (85 %, таблиця 2): помилилася на п'яти парашутах і одній собаці.

Так само модель перевірялася після кожного наступного етапу; результати зведено в таблицю 2.

Таблиця 2 – Результати перевірки моделі на етапах роботи

Етап	Режим	Зображень у наборі	Правильно на навчальних	Правильно на перевірці
Перше навчання	швидкодія	160	91 %	34 з 40
Виправлення мітки	швидкодія	161	34 %	14 з 40
Навчання заново	точність	161	99 %	38 з 40
Виправлення мітки	точність	162	100 %	40 з 40
Додано «доповнення»	точність	322	89 %	37 з 40
Optimize Model	точність	322	99 %	40 з 40

Виправлення мітки відбувається у вкладці Use: після позначки «помилковий» програма пропонує вибрати правильну мітку, додає зображення до навчального набору і повторно навчає модель. Для парашута з рисунка 10 обрано мітку «парашут». Доновчання в режимі швидкодії не виправило модель, а зіпсувало її: частка правильно розпізнаних навчальних зображень упала до 34 % (рисунок 11), а на теці «перевірка» – до 14 з 40, причому 36 зображень з 40 модель віднесла до сміттєвозів.

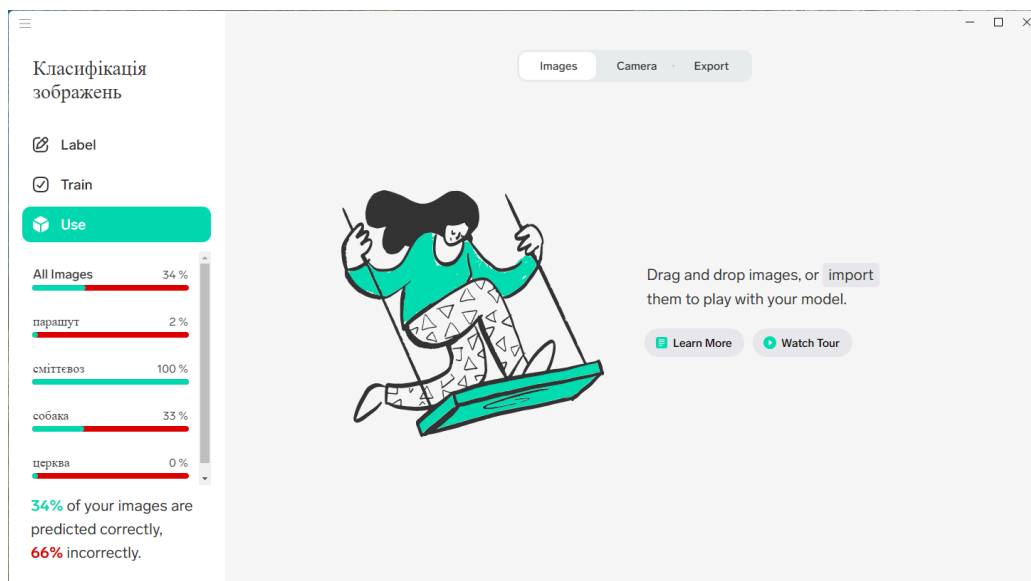


Рисунок 11 – Результат донавчання в режимі швидкодії

Тому в налаштуваннях проєкту (меню, Project Settings) обрано режим «Optimize for Accuracy» (рисунок 12). У цьому режимі Lobe використовує більшу мережу ResNet50V2; зміна режиму скидає навчання, і модель навчається заново. Навчання на відеокарті тривало 9 с. Правильно розпізнано 99 % навчальних зображень (рисунок 13) і 38 з 40 зображень теки «перевірка» (95 %).

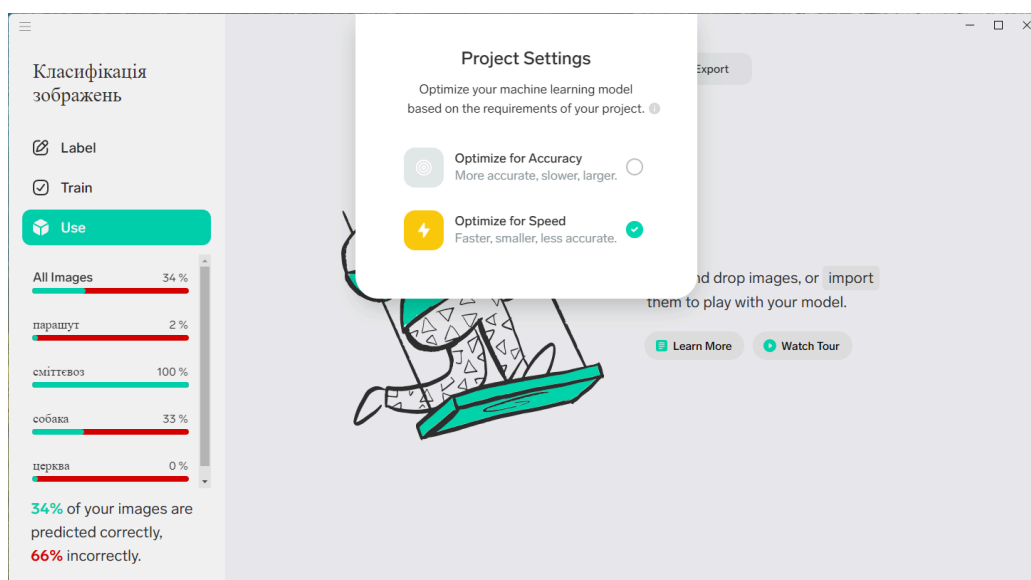


Рисунок 12 – Налаштування проєкту: вибір між точністю і швидкістю

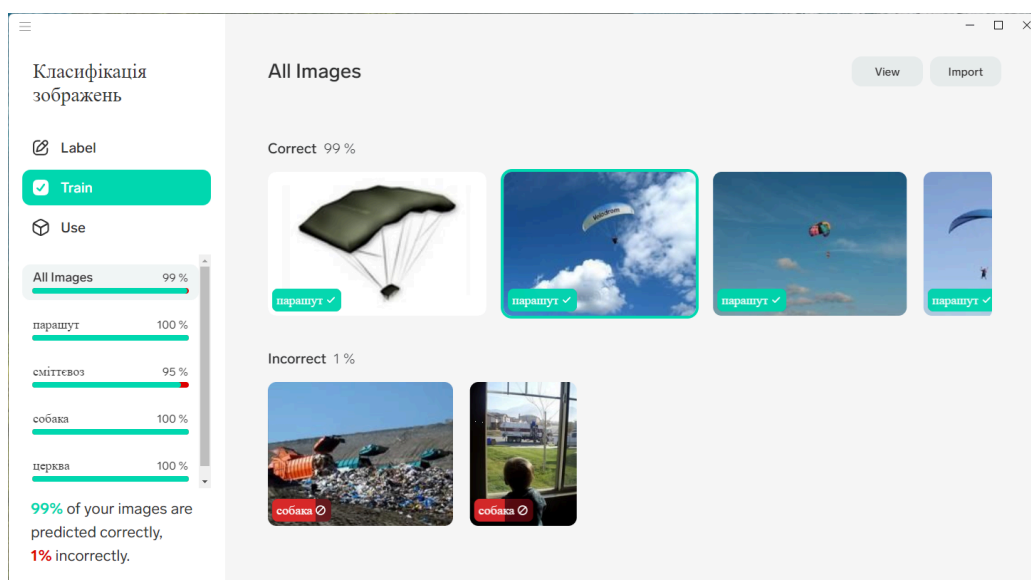


Рисунок 13 – Результат навчання в режимі точності

Обидві помилки нової моделі – парашути, прийняті за собак. На одному з цих зображень поруч із розкладеним куполом парашута справді стоїть собака (рисунок 14), тож прогноз «собака» можна пояснити; мітку виправлено на «парашут», як у вихідному наборі. Після донавчання модель правильно розпізнала всі навчальні зображення (рисунок 15) і всі 40 зображень теки «перевірка». Два з них (виправлені вручну) тепер входять до навчального набору, тому на зображеннях, яких модель не бачила, результат – 38 з 38.

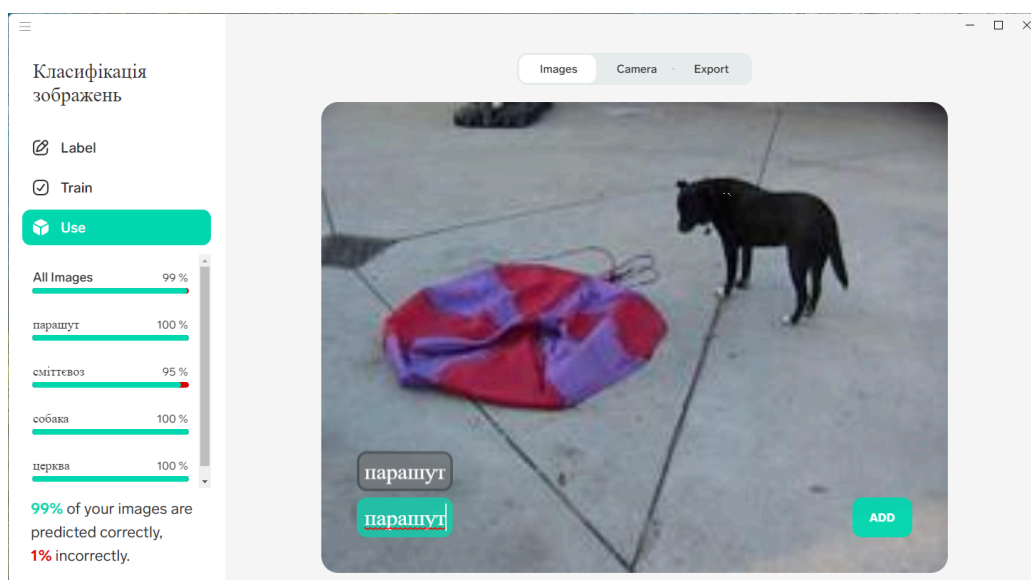


Рисунок 14 – виправлення мітки у вкладці Use

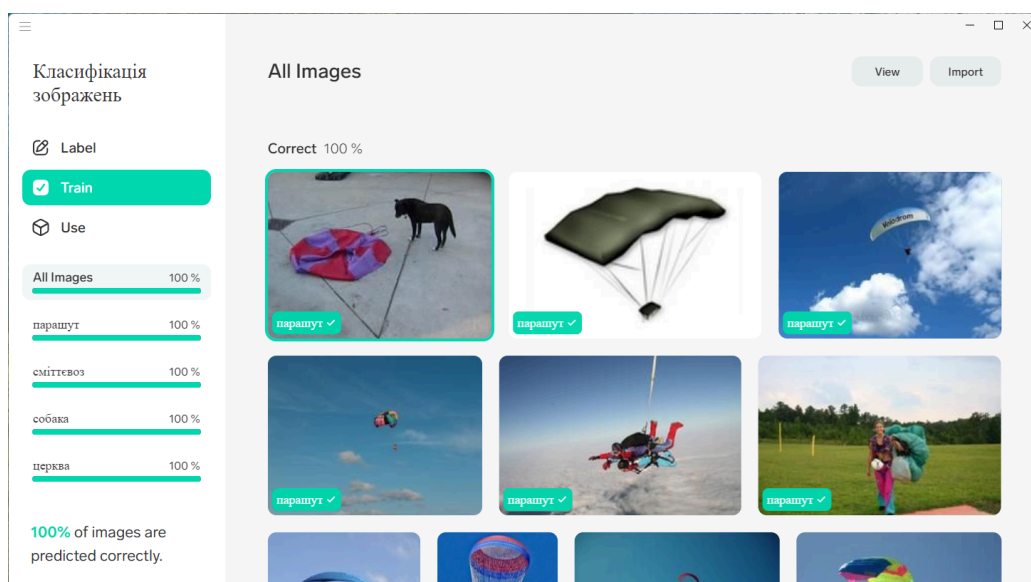


Рисунок 15 – Результат після виправлення мітки

6 ПОЛІПШЕННЯ МОДЕЛІ

Перш ніж додавати теку «доповнення», її 160 зображень подано поточній моделі як нові: правильно розпізнано 156 (97,5 %). Помилки – два парашути, прийняті за церкву, а також один сміттевоз і одна собака, прийняті за парашут.

Потім теку імпортовано так само, як навчальну: Import, Dataset, вибір теки (рисунок 16) і розмітка за назвами підтек. У навчальному наборі стало 322 зображення – по 80 на клас і ще два виправлені парашути. Lobe одразу донавчив модель, але

результат погіршився (рисунок 17): правильно розпізнано 89 % навчальних зображень, у класі «сміттєвоз» – лише 60 %, бо частину сміттєвозів, собак і церков модель віднесла до парашутів; на теці «перевірка» – 37 з 40. За журналом програми навчання зупинилося автоматично вже на п'ятій епосі, тобто модель не встигла засвоїти нові зображення.

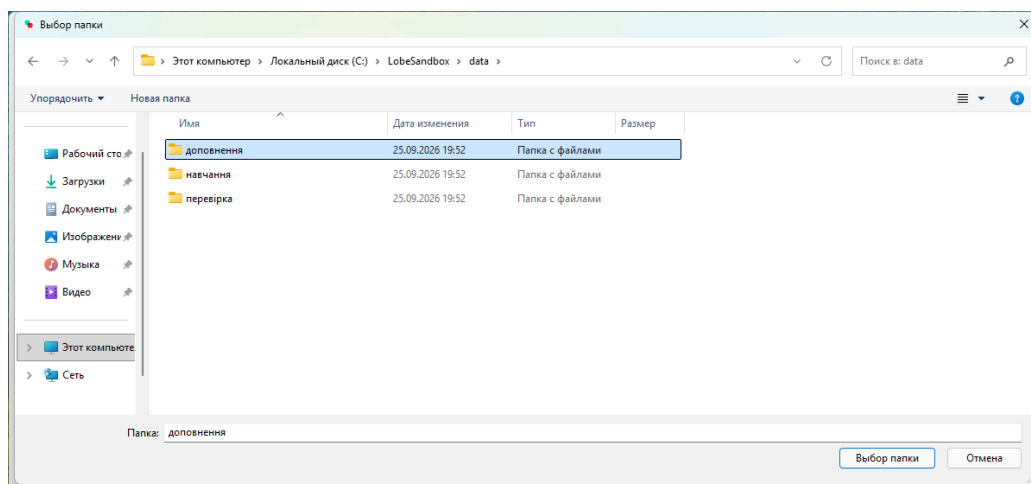


Рисунок 16 – Імпорт теки «доповнення»

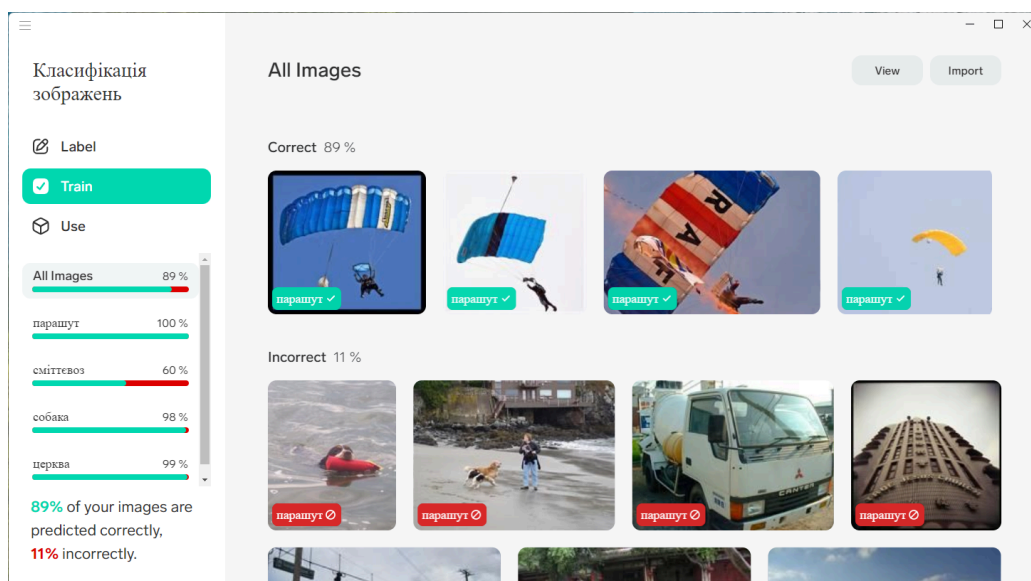


Рисунок 17 – Результат донавчання на розширеному наборі

Для подовженого навчання в меню Lobe є команда Optimize Model (рисунок 18). На відеокарті вона завершилася за 6 с. Після неї правильно розпізнано 99 % навчальних зображень (рисунок 19; помилки – три церкви й один сміттєвоз, прийняті за парашут) і всі 40 зображень теки «перевірка», з них 38 з 38 нових.

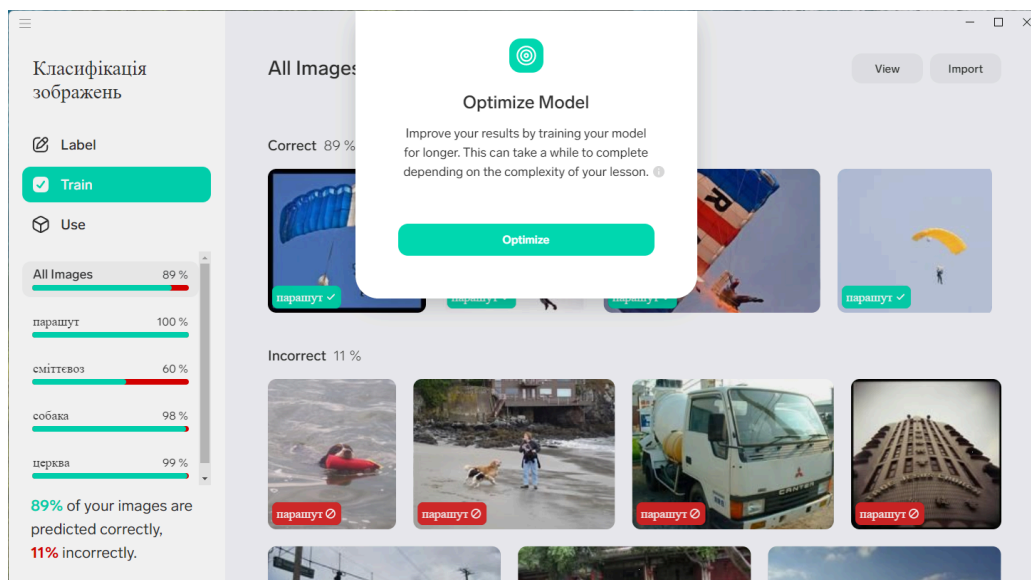


Рисунок 18 – Команда Optimize Model

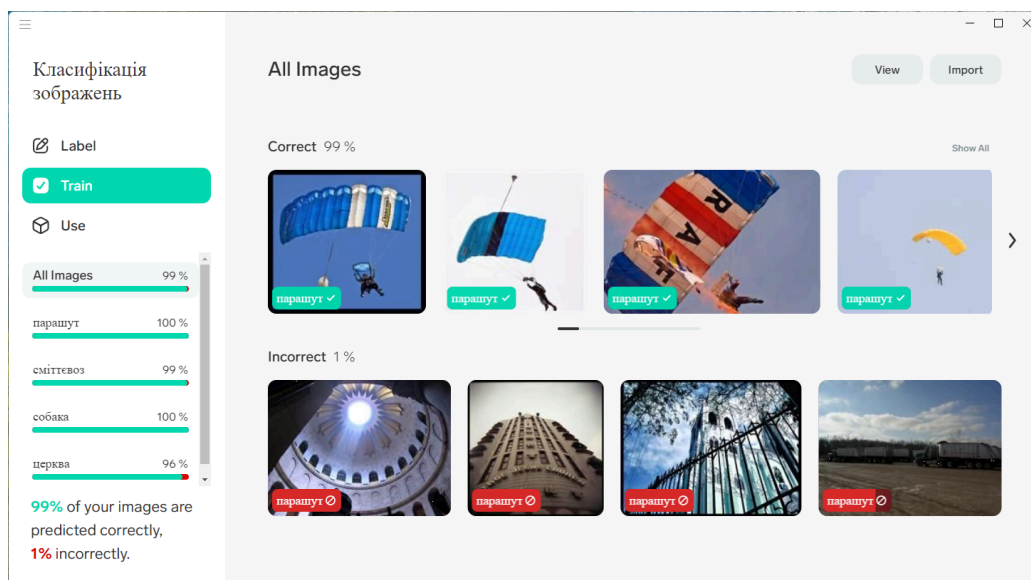


Рисунок 19 – Результат після Optimize Model

7 ЕКСПОРТ МОДЕЛІ

Експорт виконується у вкладці Use, підкладка Export. У верхній частині сторінки розташовані інтеграції (Power Platform і Lobe Connect) та шаблони застосунків на GitHub (рисунок 20), нижче – файли моделі у форматах TensorFlow, TensorFlow Lite, TensorFlow.js і ONNX (рисунок 21).

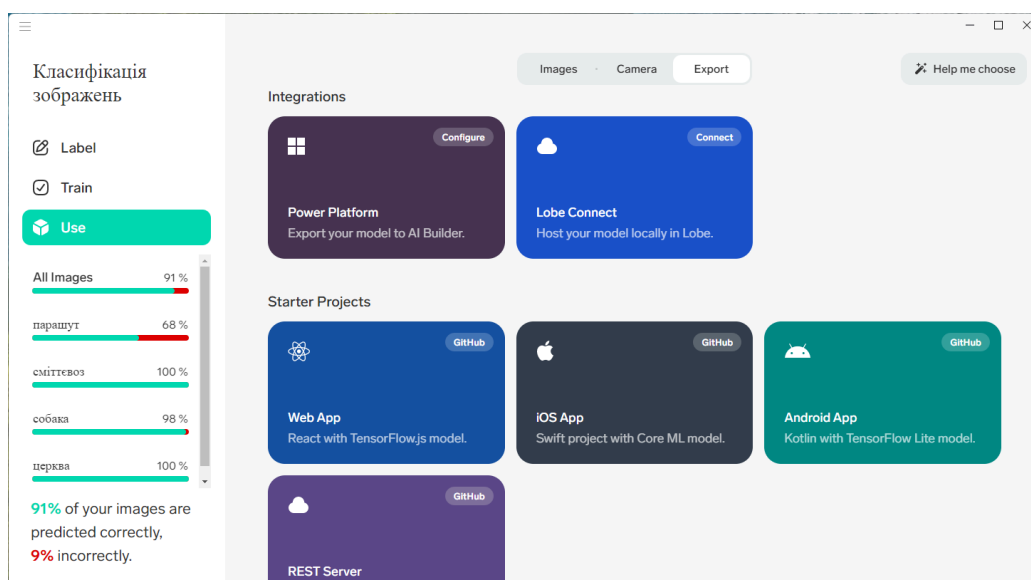


Рисунок 20 – Вкладка Export: інтеграції та шаблони застосунків

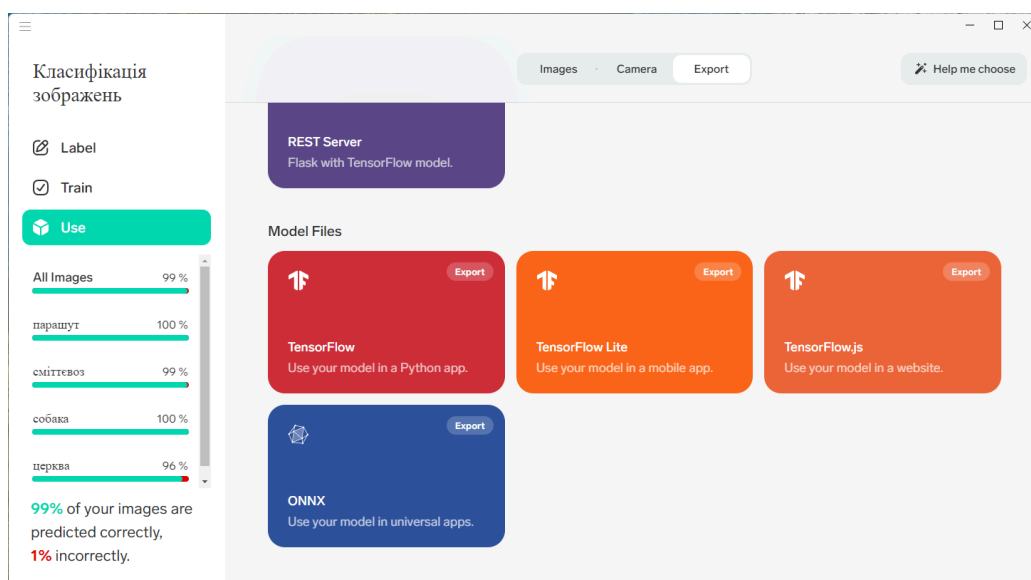


Рисунок 21 – Вкладка Export: файли моделі

TensorFlow. Після кнопки «Export» Lobe запитує теку призначення і створює в ній теку «Класифікація зображень TensorFlow» (98 МБ): модель у форматі SavedModel (saved_model.pb і тека variables), перелік міток labels.txt, опис входу й виходу signature.json та приклад використання на Python у теці example. Вхід моделі – зображення 224×224 пікселі з трьома каналами, вихід – чотири ймовірності в порядку міток «парашут», «сміттєвоз», «собака», «церква».

TensorFlow Lite. Експорт відбувається так само, але триває довше, бо модель перетворюється в компактний формат для мобільних і вбудованих пристроїв (рисунок 22). Результат – файл saved_model.tflite (95 МБ) з тими самими signature.json, labels.txt

і прикладом. Розмір обох файлів визначає мережа ResNet50V2, обрана в режимі точності.

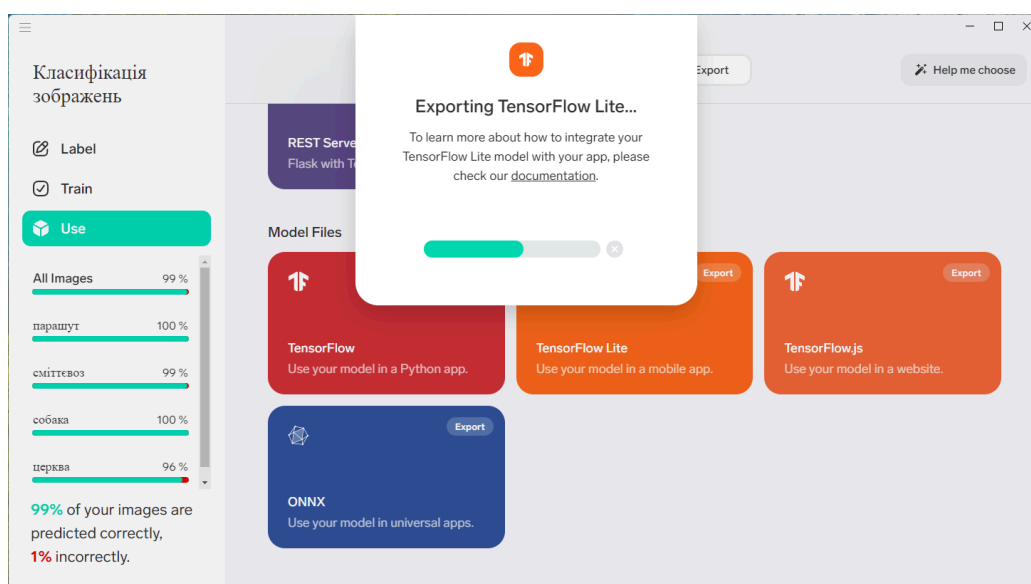


Рисунок 22 – Експорт у TensorFlow Lite

Щоб переконатися, що експортована модель працює без Lobe, файл `saved_model.tflite` запущено в Python через інтерпретатор LiteRT (бібліотека `ai-edge-litert`). Зображення готувалися так само, як у прикладі від Lobe: квадратна вирізка по центру, зменшення до 224×224 , значення від 0 до 1. Модель правильно розпізнала всі 40 зображень теки «перевірка» за 3 с на процесорі. Текст програми подається окремим файлом `lr3_check_tflite.py`.

Локальний API. Lobe Connect (рисунок 23) надає адресу `http://localhost:38101/v1/predict/` з ідентифікатором проєкту. На неї надсилається POST-запит з JSON-об'єктом, у полі `image` якого – зображення в кодуванні Base64; у відповіді повертається список прогнозів із мітками та ймовірностями. API доступний, поки відкрита програма Lobe. Саме ним у цій роботі перевірялася модель на всіх етапах (таблиця 2); сценарій перевірки подається окремим файлом `lr3_validate.ps1`.

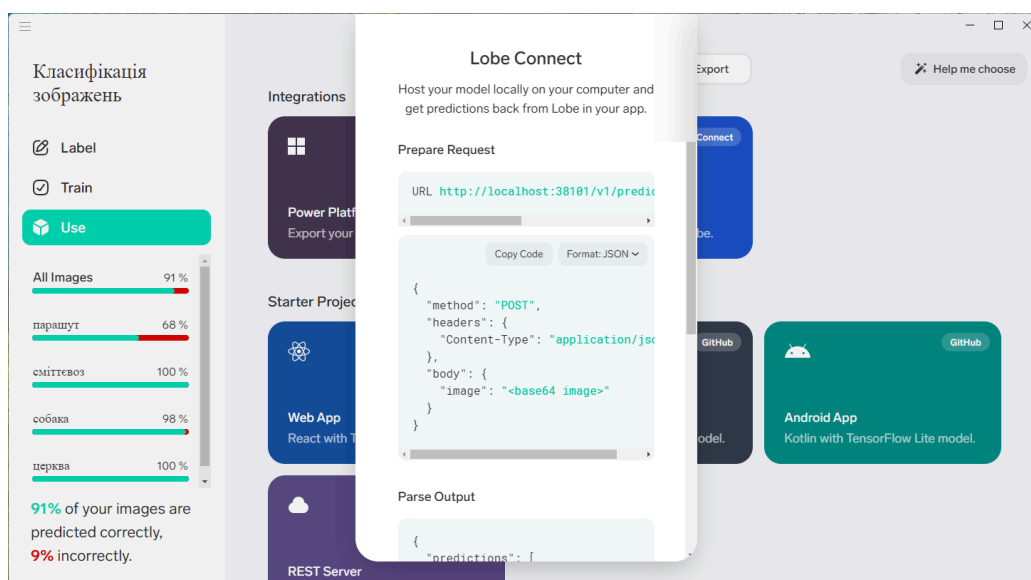


Рисунок 23 – Lobe Connect: формат запиту до локального API

ВИСНОВКИ

1. Lobe дає змогу пройти весь цикл без програмування: розмітка за назвами тек, навчання одразу після імпорту, перевірка й виправлення міток у вкладці Use, експорт у TensorFlow, TensorFlow Lite, TensorFlow.js, ONNX і локальний API.
2. У режимі швидкодії (MobileNetV2) перша модель розпізнала 91 % навчальних і 34 з 40 нових зображень; найгірше – клас «парашут». Доновчання в цьому режимі після виправлення однієї мітки зіпсувало модель: 14 з 40.
3. Режим точності (ResNet50V2) дав 99 % навчальних і 38 з 40 нових зображень, а після виправлення мітки – 38 з 38 нових. Вибір мережі вплинув на результат більше, ніж виправлення окремих прикладів.
4. Додавання 160 зображень саме по собі модель не поліпшило (89 %, 37 з 40): автоматична зупинка перервала навчання раніше, ніж модель засвоїла нові дані. Після Optimize Model – 99 % навчальних і 40 з 40 на перевірці.
5. Частка правильно розпізнаних навчальних зображень, яку показує Lobe, не замінює перевірки на нових даних. Помилки на рисунках 10 і 14 знайдено

саме на зображеннях, що не брали участі в навчанні, а перевірка всієї теки через локальний API дала кількісну оцінку кожного етапу (таблиця 2).

6. Експортована модель TensorFlow Lite працює без Lobe: у Python вона розпізнала всі 40 зображень теки «перевірка», тобто придатна для вбудовування в сторонній застосунок.